

Capítulo 1 - Building Blocks

Pregunta 1

- **Respuesta correcta:** D, E
- **Explicación:**
Para iniciar un programa en Java, la forma principal de escribir el método de entrada es `public static void main(String[] args)` (Opción E). La Opción D también funciona bien porque agregarle la palabra final a ese método es permitido en Java. Las otras opciones no sirven porque el método `main` siempre debe llevar `public`, `static` y `void`.

Pregunta 2

- **Respuesta correcta:** C, D, E
- **Explicación:**
No es obligatorio declarar el paquete (`package`) ni las importaciones (`import`). Pero si decides usarlos, debes ponerlos siempre en este orden de arriba hacia abajo: 1° paquete, 2° importaciones y 3° la clase. La Opción C tiene las tres partes en orden, la Opción D no usa paquete y la Opción E no usa importaciones, por lo que las tres son correctas.

Pregunta 3

- **Respuesta correcta:** A, E
- **Explicación:**
Al escribir `public class Bunny`, el nombre `Bunny` sirve para darle nombre a la clase (Opción A). Al poner `Bunny bun = new Bunny();`, la variable `bun` es el objeto que se crea en memoria (Opción E). En el método `main`, no importa si al parámetro de texto le llamas `x`, `args` u otro nombre, siempre que sea de tipo `String[]`.

Pregunta 4

- **Respuesta correcta:** B, E, G
- **Explicación:**
Un nombre de variable o clase en Java puede llevar letras, números, `( $ )` y `( _ )`, pero nunca debe empezar con un número ni ser una palabra reservada del lenguaje. Por eso `_helloWorld$` (B), `Public` con mayúscula (E) y `_Q2_` (G) son válidos. Un guion bajo solo `( _ )` o palabras como `true` no están permitidos.

Pregunta 5

- **Respuesta correcta:** A, D, F
- **Explicación:**
Nunca sabemos exactamente cuándo se ejecutará el limpiador de memoria o recolector de basura (`Garbage Collector`) (Opción F). El primer objeto `Bear` sigue usándose hasta la línea 13 (Opción A). El segundo objeto `Bear` se queda sin nadie que lo use justo después de la línea 13 al asignar `null` a la variable, por lo que queda listo para eliminarse (Opción D).

Pregunta 6

- **Respuesta correcta:** F
- **Explicación:**

Las variables solo existen dentro del bloque de llaves {} donde fueron creadas. En la línea 14, las variables que estaban dentro de bloques más pequeños ya desaparecieron, pero siguen existiendo las 3 variables de la clase y las 4 variables activas del método (distance, path, water, age e i), lo que da un total de 7 variables disponibles.

Pregunta 7

- **Respuesta correcta:** C, E
- **Explicación:**

En los bloques de texto creados con triple comilla ("""), todo lo que escribes adentro se imprime tal cual. Como no calcula variables ni ejecuta código, imprime directamente el texto # cups = 0 (Opción C) y mantiene los espacios o sangría del inicio (Opción E).

Pregunta 8

- **Respuesta correcta:** B, D, E, H
- **Explicación:**

Al usar var, Java necesita que le des un valor inmediato para adivinar el tipo de dato. La Opción B detecta texto (String), la D un número entero (Integer), la E hace una operación matemática válida y la H detecta texto y luego permite cambiarlo a null. No puedes asignarle null directamente desde el inicio ni declarar varias variables en la misma línea.

Pregunta 9

- **Respuesta correcta:** E
- **Explicación:**

Las variables que creas dentro de un método debes darle un valor inicial antes de usarlas. En cambio, las variables de una clase (como String) se inicializan solas con el valor por defecto null si no les pones nada (Opción E).

Pregunta 10

- **Respuesta correcta:** A, E, F
- **Explicación:**

Java permite usar guiones bajos (_) en los números para que sean más fáciles de leer, pero únicamente si están colocados entre dos dígitos. Por eso 3_1 (A), 2_234.0_0 (E) y 9__6 (F) son válidos. No puedes poner guiones bajos al principio, al final o al lado de un punto decimal.

Pregunta 11

- **Respuesta correcta:** E

- **Explicación:**

Las dos primeras importaciones de java.lang no se necesitan porque Java siempre incluye esas funciones automáticamente. Las dos importaciones de aquarium tampoco son necesarias porque la clase ya está dentro del paquete aquarium. Por eso, las 4 importaciones se pueden borrar sin causar ningún problema.

Pregunta 12

- **Respuesta correcta:** A, C, D

- **Explicación:**

La línea 2 da error porque no puedes declarar dos tipos diferentes de variables (int y double) en la misma línea separadas por comas (Opción A). La línea 4 falla porque en Java no se pueden dar valores por defecto a las variables dentro de los parámetros (Opción C). La línea 7 falla porque la variable fins se creó en otro bloque de llaves y ya no existe ahí (Opción D).

Pregunta 13

- **Respuesta correcta:** A, B, C

- **Explicación:**

La Opción A importa todas las clases del paquete aquarium usando *. Las Opciones B y C traen directamente la clase Water por su nombre, lo cual evita confusiones con clases de otros paquetes que se llamen igual. La Opción D causa confusión al traer clases repetidas con * y la E tiene una línea duplicada que da error.

Pregunta 14

- **Respuesta correcta:** A, B, D, E

- **Explicación:**

La línea 3 da error porque un valor long (5L) es muy grande para guardarlo en un short sin indicarlo explícitamente (Opción A). La línea 4 falla porque un número con decimales double (2.0) no cabe en un entero int (Opción B). Las líneas 6 y 7 fallan porque no se pueden usar métodos ni propiedades en tipos simples o primitivos (Opciones D y E). La línea 8 sí funciona bien porque usa .length() en una cadena de texto (String).

Pregunta 15

- **Respuesta correcta:** C, E, F

- **Explicación:**

La recolección de basura borra de la memoria los objetos que ya nadie está usando (Opción C). Si el programa se cierra rápido, puede que el recolector ni siquiera alcance a ejecutarse (Opción E). Para que un objeto pueda borrarse, la regla principal es que ninguna variable esté apuntando a él (Opción F).

Pregunta 16

- **Respuesta correcta:** A, D

- **Explicación:**

En los bloques de texto, la barra invertida (\) al final de la línea evita el salto de línea y junta el texto, haciendo que todo se imprima en solo dos líneas (Opción A). El símbolo \s sirve para decirle a Java que mantenga los espacios en blanco del final (Opción D).

Pregunta 17

- **Respuesta correcta:** D, F, G

- **Explicación:**

Una variable boolean de una clase que no tiene valor inicia por defecto como false, mostrando Empty = false (Opción D). Una variable de texto String sin valor inicia como null, mostrando Brand = null (Opción F). Los números con decimales float sin valor inicial se muestran como 0.0 (Opción G).

Pregunta 18

- **Respuesta correcta:** B, C, F

- **Explicación:**

Al declarar con var, Java identifica el tipo de dato mientras se compila el código (Opción B). No se permite usar var en los parámetros de los métodos ni en las variables principales de una clase (Opción C). El tipo de dato no se puede cambiar después, pero el valor que contiene la variable sí se puede modificar (Opción F).

Pregunta 19

- **Respuesta correcta:** A, D

- **Explicación:**

El método Long.parseLong() convierte un texto en un número entero simple long, mientras que Long.valueOf() convierte el texto en un objeto Long (Opción D). El código funciona correctamente y realiza las operaciones hasta imprimir el valor final de 100 (Opción A).

Pregunta 20

- **Respuesta correcta:** C

- **Explicación:**

Al poner public void PoliceBox(), se creó un método normal y no un constructor debido a la palabra void. Por eso, al crear los objetos, las variables color y age se quedan con sus valores vacíos por defecto (null y 0L). Al hacer p = q, la variable p pasa a mirar al mismo objeto que q, imprimiendo P1=null (Opción C).

Pregunta 21

- **Respuesta correcta:** D

- **Explicación:**

El programa empieza en main e imprime 7-. Al crear new Salmon(), primero ejecuta las instrucciones del objeto en orden: imprime 0- (valor inicial) y sube a 1. Luego se

ejecuta el constructor, que cambia el valor a 4 e imprime 2-. Por último, imprime la variable en main (4-), formando el resultado 7-0-2-4- (Opción D).

Pregunta 22

- **Respuesta correcta:** C, F, G

- **Explicación:**

Los números expresados en hexadecimal (0xE = 14) y en binario (0b101 = 5) son enteros válidos en Java (Opciones C y F). La Opción G también es correcta porque los guiones bajos están bien puestos entre los números. Las demás fallan por combinar tipos de datos incompatibles o colocar mal los guiones bajos.

Pregunta 23

- **Respuesta correcta:** A, D

- **Explicación:**

La línea 3 da error de compilación porque intenta guardar un número decimal double (50.0) en una variable float sin ponerle la letra f al final (Opción A). La línea 10 falla porque la variable depth se creó dentro del bucle for y ya no se puede usar fuera de sus llaves (Opción D).

Capítulo 2 - Operators

Pregunta 1

- **Respuesta correcta:** A, D, G
- **Explicación:**
== es el operador de igualdad y se aplica a primitivos y referencias de objetos. ! es el operador de complemento lógico que se usa exclusivamente con tipos boolean. (boolean) es un casteo válido puesto que boolean es un tipo de dato reconocido en Java. Los demás operadores (+, -, %, ~) son puramente numéricos o de bits.

Pregunta 2

- **Respuesta correcta:** A, B, D
- **Explicación:**
Por la regla de promoción numérica, al aplicar el operador binario + entre tipos enteros menores como byte y short, ambos se promueven automáticamente a int. Por lo tanto, el resultado es un int y solo se puede asignar sin casteo explícito a variables de tipo int, long o double.

Pregunta 3

- **Respuesta correcta:** B, C, D, F
- **Explicación:**
La multiplicación 2 * ear se promueve automáticamente a long y no se puede guardar directamente en la variable hearing de tipo int. Cada opción resuelve el problema de forma independiente: castear ear a int (B), definir ear como short (C), castear el resultado (int)(2 * ear) (D), o cambiar la variable hearing a long (F).

Pregunta 4

- **Respuesta correcta:** B
- **Explicación:**
La expresión (teeth != 10) evalúa a true. La reasignación (wolf = false) le otorga el valor false a wolf y retorna false. Al evaluar true ^ false (XOR), el resultado es true, el cual se asigna a canine mientras que teeth no sufre cambios y permanece en 20.

Pregunta 5

- **Respuesta correcta:** A, C
- **Explicación:**
En la opción A, la adición + tiene menor precedencia que los multiplicativos * y %, los cuales tienen menor o igual precedencia que el operador unario -. En la opción C, el operador de asignación = tiene la precedencia más baja, seguido del operador de igualdad == y la negación unaria !.

Pregunta 6

- **Respuesta correcta:** F

- **Explicación:**

Por reglas de precedencia, el casteo (int) se aplica de forma aislada a la variable fruit. Al sumarse con vegetables (de tipo float), la expresión completa se promueve a float, impidiendo que el método retorne un valor long sin un casteo explícito de toda la suma.

Pregunta 7

- **Respuesta correcta:** D

- **Explicación:**

clear: $2 > 1 \ \& \ (\text{true} \ || \ \text{false})$ evalúa a true.

safe: Como $(\text{vis} > 2)$ es false, el operador $\&\&$ realiza cortocircuito y omite la evaluación de $(\text{ph}++ > 1)$. ph se mantiene en 7 y safe es false.

tasty: El pre-decremento $--\text{ph}$ reduce ph a 6. Luego $7 \leq 6$ evalúa a false.

Pregunta 8

- **Respuesta correcta:** A

- **Explicación:**

En `pig = pig++;`, al usar el post-incremento, la expresión devuelve el valor original (4) para la asignación y la actualización a 5 se descarta. En `goat -= 1.0;`, los operadores compuestos aplican un casteo implícito automático al tipo de la variable destino (long), reduciendo goat de 2 a 1.

Pregunta 9

- **Respuesta correcta:** A, D, E

- **Explicación:**

Línea 1: $a > 2$ ($2 > 2$) es false, evalúa `b++` e imprime 4 (D).

Línea 2: $a != c$ ($2 != 2$) es false, evalúa `b++` (con b en 5), retornando 5 que se asigna nuevamente a b (E).

Línea 3: $a > b$ ($2 > 5$) es false, la rama final del ternario evalúa a 1 (A).

Pregunta 10

- **Respuesta correcta:** G

- **Explicación:**

La línea 2 no compila. Aunque weight y height se castean individualmente a byte, la operación binaria de multiplicación `*` promueve automáticamente el resultado a int. Intentar almacenar ese resultado int en la variable short zebra provoca un error de compilación.

Pregunta 11

- **Respuesta correcta:** D

- **Explicación:**

sample1: $(8) \% 3 = 2$.

sample2: De izquierda a derecha, $3 * 2 \% 3 = 6 \% 3 = 0$.

sample3: $5 * (1 \% 2) = 5 * 1 = 5$.

Pregunta 12

- **Respuesta correcta:** D

- **Explicación:**

El operador post-increment incrementa el valor de la variable pero devuelve el valor original, mientras que el operador pre-decrement decrementa la variable y devuelve el valor nuevo ya modificado.

Pregunta 13

- **Respuesta correcta:** F

- **Explicación:**

goingToTheStore: true & false ^ true = false ^ true = true.

goingToTheZoo: true && !false = true.

stayingHome: !(true && true) = false.

Pregunta 14

- **Respuesta correcta:** B, E, G

- **Explicación:**

El operador != se utiliza para comparar referencias entre objetos (B). El valor de retorno de una expresión de asignación es el propio valor reasignado a la variable (E). El complemento lógico ! actúa únicamente sobre booleanos y no invierte signos numéricos (G).

Pregunta 15

- **Respuesta correcta:** D

- **Explicación:**

El operador ternario (? :) es el único operador en Java que requiere exactamente tres operandos para funcionar.

Pregunta 16

- **Respuesta correcta:** B

- **Explicación:**

Únicamente la línea 1 contiene un error de compilación. La expresión (long)3 promueve la suma entera a long, impidiendo que se asigne a la variable int note sin un casteo explícito. Las demás líneas utilizan casteos adecuados y asignación compuesta válida.

Pregunta 17

- **Respuesta correcta:** C, F

- **Explicación:**

En ticketsSold += 1 + ticketsTaken++;, ticketsTaken++ retorna 1 (y sube a 2), dejando ticketsSold en 3 + 1 + 1 = 5. Luego ticketsTaken *= 2; incrementa ticketsTaken a 4 (F). Finalmente ticketsSold += (long)1; (con casteo implícito en la asignación compuesta) sube ticketsSold a 6 (C).

Pregunta 18

- **Respuesta correcta:** C
- **Explicación:**

En Java, únicamente los paréntesis () son caracteres válidos para agrupar expresiones y alterar manualmente la precedencia de los operadores.

Pregunta 19

- **Respuesta correcta:** B, F
- **Explicación:**

En la línea 5, ++start incrementa start a 8 y lo suma a end (4 + 8), asignando **12** a end (F). En la línea 6, Byte.MAX_VALUE es 127; al sumarle 1 (128) y castear a byte, ocurre un desbordamiento numérico (overflow), pasando a **-128** (B).

Pregunta 20

- **Respuesta correcta:** A, D, E
- **Explicación:**

Los operadores unarios tienen un nivel de precedencia superior frente a los operadores binarios aritméticos o ternarios (A). El operador post-decremento devuelve el valor previo a la modificación (D). El operador ! no se puede aplicar a valores numéricos (E).

Pregunta 21

- **Respuesta correcta:** E
- **Explicación:**

bird calcula el complemento de bits ~8, que equivale a -8 - 1 = -9. plane calcula -8. La comparación bird == plane (-9 == -8) resulta en false, por lo que superman recibe el valor 10. Finalmente, --superman decrementa la variable antes de imprimir, dando como resultado **9**. Imprime -9,-8,9 (E).